



**Universidad
Europea** MADRID

PROYECTO INTEGRADOR

1DAW

Los Hijos Del Compilador

[LOGO]

Diego Muñoz Aviles

David Martinez del Rio

Javier Garcia Pablos

Roberto Canto Pedrera

Índice

Resumen	2
1. Introducción	3
2. Objetivos	3
3. Tecnologías utilizadas	3
4. Desarrollo e implementación	3
5. Metodología	3
6. Resultados y conclusiones	4
7. Trabajos futuros	4
Anexos	5
Anexo I – Listado de requisitos de la aplicación	5
Anexo II – Guía de uso de la aplicación	5
Anexo III	5

1. Introducción:

Este documento recoge el trabajo realizado durante el tercer sprint del Proyecto Integrador. Tras haber definido en los sprints previos el análisis funcional del sistema, la capa de Vista con las ventanas Swing, el modelo relacional y la creación física de la base de datos, en este Sprint 3 se afronta la construcción de la capa de Modelo y la capa de Control que dan vida a la aplicación. Se trata por tanto del sprint en el que el proyecto deja de ser un conjunto de prototipos inconexos y empieza a funcionar como una aplicación integrada siguiendo el patrón MVC.

El objetivo general del sprint es doble. En primer lugar, traducir el diagrama de clases UML, creando las clases del modelo con sus atributos, operaciones y asociaciones. En segundo lugar, crear las clases del control que actúan como puente entre la vista y el modelo, encapsulando tanto el acceso a la base de datos MySQL mediante JDBC como las reglas de negocio descritas en el enunciado.

La tarea concreta dentro de este sprint es la [PR4], que establece literalmente:

- > - A partir del diagrama de clases:
- > - Creación de las clases pertenecientes al Modelo.
- > - Creación de las clases pertenecientes al Control encargadas del acceso a BBDD y del manejo de la aplicación.

Paralelamente, la tarea [ED9] requiere redactar los apartados Introducción y Desarrollo de la memoria final del proyecto, que son los que se recogen a continuación.

Los módulos implicados en este sprint son los tres del ciclo (Programación, Bases de Datos y Entornos de Desarrollo), ya que se combina diseño orientado a objetos, consultas SQL y documentación/UML.

2. Desarrollo:

2.1 Planificación del Sprint:

Siguiendo la metodología SCRUM adoptada por el equipo, el sprint se ha planificado a partir del tablero Trello dividiendo el trabajo en cinco **user stories** técnicas:

1. Traducir el diagrama de clases al Modelo Java.
2. Implementar la conexión JDBC única a la BBDD `edna moda`.
3. Implementar los DAO (Data Access Object) con el CRUD completo.
4. Implementar los Controladores con la lógica de negocio y los permisos por rol.
5. Escribir una clase `MainApp` que integre la Vista del Sprint 2 con el Control del Sprint 3.

Cada historia se ha trabajado en una rama `feature/...` de GitHub y se ha integrado en `developed` al completarse, de manera que siempre queda una línea de código estable al final del sprint.

2.2 Estructura del código:

La nueva carpeta `src/TercerSprint` sigue el patrón ****MVC**** y queda organizada así:

src/

- |— PrimerSprint
- |— SegundoSprint
- | |— vista
- | |— bd/edna_moda.sql (Sprint 2 - Script SQL)
- |— TercerSprint
- |— modelo
- | |— Empleado.java
- | |— Cliente.java
- | |— Traje.java
- | |— Taller.java
- | |— Cita.java
- |— control
- | |— ConexionBD.java
- | |— SesionActual.java
- | |— EmpleadoDAO.java
- | |— ClienteDAO.java
- | |— TrajeDAO.java
- | |— TallerDAO.java
- | |— CitaDAO.java
- | |— LoginControlador.java
- | |— CitasControlador.java

```
| └─ ClientesControlador.java  
| └─ TalleresControlador.java  
└─ MainApp.java
```

2.3 Capa Modelo:

Se ha creado una clase por cada entidad del diagrama de clases, y los atributos se han puesto usando las clases estándar de Java cuando procedía. Los valores restringidos del diagrama categoría del empleado, estado del traje, tipo de la sala y tipo del cliente se han implementado con métodos `fromTexto()` que permiten convertir de forma segura las cadenas provenientes de MySQL a los valores del enum y viceversa.

Las operaciones que aparecen en el diagrama (`asignarEmpleado()`, `cambiarHorario()`, `cambiarEstado()`, `modificarTaller()`) se han incluido en las clases correspondientes. Además se han añadido pequeños métodos de ayuda que facilitan el código del control, como `Empleado.puedeCrearCitas()`, `Empleado.esMaestro()` y `Cita.getHoraFin()`.

En la clase `Cita` se guarda la lista de aprendices como una `List<Integer>`, en coherencia con la tabla N:M cita aprendices de la BBDD. La propia clase conoce la regla “máximo 2 aprendices” (constante `MAX APRENDICES = 2`) y la valida en el método `anadirAprendiz()`.

2.4 Capa Control - Acceso a BBDD:

El acceso a la base de datos se ha centralizado en la clase `ConexionBD`, que actúa como fábrica de conexiones JDBC al servidor MySQL. La URL y las credenciales están parametrizadas y pueden sobrescribirse para los equipos que tengan configuraciones distintas en local. Todas las conexiones se abren y cierran dentro de bloques, garantizando que no quedan recursos colgados aunque se produzcan excepciones SQL.

Sobre esta conexión se han construido los DAO (`EmpleadoDAO`, `ClienteDAO`, `TrajeDAO`, `TallerDAO`, `CitaDAO`). Son los únicos objetos que contienen sentencias SQL, y todas ellas usan `PreparedStatement` para evitar inyección SQL y mejorar el rendimiento. Cada DAO incluye un método privado `map(ResultSet)` que convierte una fila SQL en su objeto del modelo, lo que hace el resto del CRUD más legible.

Hay tres puntos del acceso a datos que merecen mención especial:

- **Autenticación.**
- **Transacciones**
- **Consulta para reglas de negocio**

2.5 Capa Control - Lógica de la Aplicación:

Por encima de los DAO están los controladores, que son los que usan las vistas. Esta separación (DAO = datos / Controlador = negocio) facilita el testing y cumple el objetivo del módulo de Programación de implementar correctamente el patrón MVC.

- `SesionActual` mantiene el empleado autenticado como singleton, de modo que cualquier controlador puede consultar el rol activo.
- `LoginControlador` sustituye al "TODO" que quedaba en la `LoginVista` del Sprint 2. Conecta el botón ****Entrar**** con `EmpleadoDAO.autenticar(...)`, inicia la `SesionActual`, cierra la ventana de login y abre la `VentanaPrincipal` pasándole el apodo y la categoría (lo que activa los permisos de menú ya preparados en la vista).
- `CitasControlador` concentra las reglas más complejas del enunciado:
 - Solo Oficiales y Maestros pueden crear citas.
 - Un Oficial únicamente puede modificar las citas de las que es responsable (lo comprueba contra la BBDD antes de disparar el `UPDATE`).
 - Se valida la duración mínima (1 h) y el número máximo de aprendices (2).
 - Se comprueba el ****solape horario**** en el mismo taller y día.
 - Se aplica el ****BONUS**** descrito en el enunciado: si el cliente de la nueva cita es héroe (o villano) y ya hay una cita en ese taller y día para un cliente del tipo contrario, debe haber al menos 60 minutos de margen entre ambas.
- `ClientesControlador` y `TalleresControlador` implementan los permisos descritos en el documento: el Aprendiz no accede, el Oficial solo consulta talleres, y el Maestro tiene acceso total. Estas reglas se centralizan en métodos privados `exigirMaestro()`, que lanzan `IllegalStateException` con un mensaje claro cuando no se cumplen — las vistas lo convierten en un `JOptionPane` de error.

2.6 Dificultades encontradas y decisiones tomadas:

Durante el sprint han surgido varias decisiones de diseño que merece la pena comentar:

- **Conexión por operación vs. conexión única.** Se ha optado por abrir y cerrar una conexión por operación (try-with-resources) en lugar de mantener una conexión global. Es ligeramente menos eficiente, pero mucho más robusto y evita tener que reconectar manualmente cuando MySQL cierra sesiones ociosas.
- **Contraseñas hasheadas.** Al usar SHA-256 en la aplicación y `SHA2(...,256)` en la BBDD conseguimos un esquema simple pero realista, evitando guardar contraseñas en claro. Para un entorno de producción real habría que añadir sal y un algoritmo adaptativo (bcrypt), pero queda fuera del alcance del proyecto.

- **Validación de reglas en el Controlador y no en el DAO.** Esto mantiene los DAO limpios y reutilizables, y permite testear las reglas de negocio sin necesidad de BBDD.

- **Sin cambios en la Vista.** Todas las vistas del Sprint 2 se siguen usando tal cual, siguiendo el principio del MVC: un cambio en la capa de control no debería afectar a la interfaz. Las vistas solo se tocarán en el Sprint 4 para refinar la interacción con el usuario.

2.7 Trabajo pendiente para el Sprint 4:

Al cerrar el Sprint 3 queda:

- Conectar los listeners de las ventanas `CitasVista`, `ClientesVista` y `TalleresVista` con los controladores correspondientes ([PR5] Refinamiento del programa).
- Redactar las pruebas JUnit de los DAO y controladores ([ED10]).
- Documentar todas las clases con JavaDoc ([ED10]).
- Escribir el manual de usuario en la wiki de GitHub ([ED11]).

Con esto, al finalizar el sprint el equipo dispone de una aplicación que ya se puede arrancar, autenticar un empleado real contra la BBDD, y que deja toda la lógica de negocio lista para que la interfaz simplemente la invoque en el siguiente sprint.